

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI  
PHẦN HIỆU TẠI TP.HCM  
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



# Quy trình Kiểm thử Phần mềm

Buổi 2 / 12 — Môn: Kiểm thử Phần mềm

Ngày 16 tháng 8 năm 2026

# Nội dung buổi học

- ➊ Vị trí trong đề cương — mục tiêu
- ➋ Một số quy trình phát triển phần mềm: Waterfall, V-model, Iterative, Agile/Scrum
- ➌ Quy trình kiểm thử phần mềm (STLC): 6 giai đoạn
- ➍ Các mức kiểm thử: unit → acceptance trên DigiShield
- ➎ Các loại kiểm thử; re-test và regression
- ➏ Testing Pyramid
- ➐ Test Plan (IEEE 829), Test Strategy, traceability matrix
- ➑ Vai trò trong nhóm kiểm thử; defect lifecycle
- ➒ **Giao Bài tập lớn (30%)**
- ➓ Thực hành và bài nộp

# Vị trí trong đề cương

## Chương 2 — Quy trình kiểm thử phần mềm

- 2.1. Giới thiệu
- 2.2. Một số quy trình phát triển phần mềm
- 2.3. Quy trình kiểm thử phần mềm

## Kết nối với các buổi khác

Buổi 1 (C1) trả lời “*kiểm thử là gì, tại sao*”. Buổi này trả lời “*kiểm thử diễn ra **khi nào, ở đâu** trong dự án và **theo quy trình nào***”. Từ Buổi 3 trở đi ta đi vào **kỹ thuật** thiết kế và cài đặt test.

## Thông báo quan trọng hôm nay

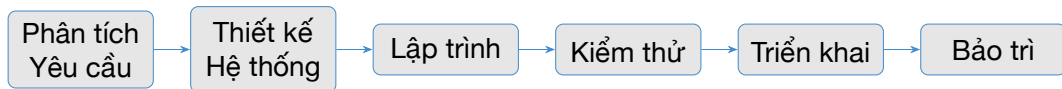
Giao **Bài tập lớn (30% điểm học phần)** và thông báo **bài kiểm tra viết Chương 1+2 (10%)** vào đầu Buổi 3.

# Mục tiêu buổi học

Sau buổi học, sinh viên có thể:

- Mô tả vị trí của hoạt động kiểm thử trong các mô hình phát triển: Waterfall, V-model, Iterative/Incremental, Agile/Scrum.
- Trình bày 6 giai đoạn của STLC kèm entry/exit criteria từng giai đoạn.
- Phân biệt 4 mức kiểm thử và **ánh xạ được vào DigiShield**: lệnh nào, thư mục nào, công cụ nào cho mỗi mức.
- Phân biệt kiểm thử chức năng/phi chức năng, re-test/regression.
- Soạn được một Test Plan rút gọn theo IEEE 829 và một traceability matrix.
- Hiểu rõ yêu cầu, timeline và rubric của Bài tập lớn.

# SDLC — Vòng đời phát triển phần mềm



## Câu hỏi trung tâm của Chương 2

Giai đoạn “Kiểm thử” nằm **ở đâu**? Câu trả lời **khác nhau** tùy mô hình phát triển: Waterfall đặt kiểm thử ở *cuối*, V-model đặt nó *đối xứng* với từng pha phát triển, Agile *rải đều* nó vào mọi Sprint.

Bốn mô hình sẽ xét: **Waterfall, V-model, Iterative/Incremental, Agile/Scrum.**

# Mô hình Waterfall (thác nước)

## Đặc điểm

- Tuyến tính: mỗi giai đoạn hoàn tất mới sang giai đoạn kế
- Tài liệu hóa nặng, ký duyệt từng pha
- Kiểm thử là **một pha riêng, sau lập trình**

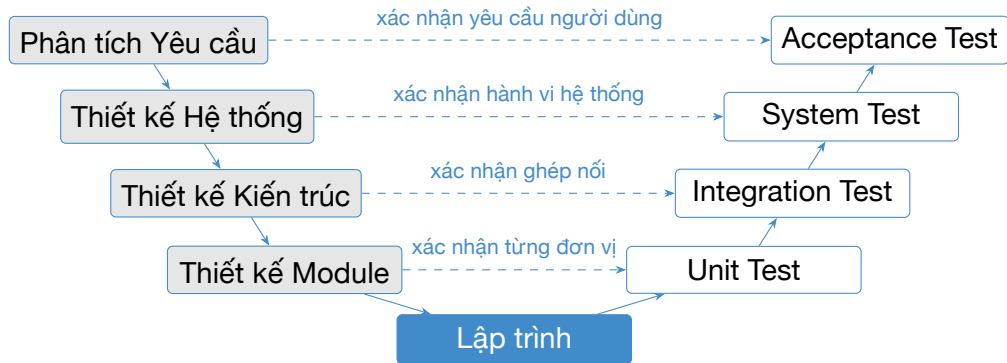
## Phù hợp khi

- Yêu cầu ổn định, ít thay đổi
- Dự án có ràng buộc hợp đồng/pháp lý chặt

## Hệ quả với kiểm thử

- Lỗi yêu cầu/thiết kế bị phát hiện **rất muộn** — chi phí sửa cao (quy tắc 1:10:100, Buổi 1)
- Kiểm thử bị **đồn ép** cuối dự án: trễ tiến độ thì cắt thời gian test
- Tester tham gia muộn, ít hiểu nghiệp vụ

# Mô hình V (V-Model)



Mỗi giai đoạn **phát triển** (trái) có một mức **kiểm thử** tương ứng (phải); test được *thiết kế ngay khi có tài liệu pha trái*.

# Đọc hiểu mô hình V

| Pha phát triển (trái) | Mức kiểm thử (phải) | Test basis                     |
|-----------------------|---------------------|--------------------------------|
| Phân tích yêu cầu     | Acceptance test     | Đặc tả yêu cầu, hợp đồng       |
| Thiết kế hệ thống     | System test         | Đặc tả chức năng hệ thống      |
| Thiết kế kiến trúc    | Integration test    | Thiết kế giao diện giữa module |
| Thiết kế module       | Unit test           | Đặc tả chi tiết module         |

## Ưu điểm

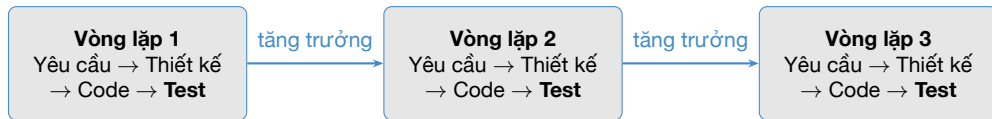
- Kiểm thử được **lập kế hoạch sớm** (nguyên tắc 3 — kiểm thử sớm)
- Mỗi sản phẩm trung gian đều có mức test kiểm chứng

## Nhược điểm

- Vẫn tuyến tính, kém thích ứng khi yêu cầu đổi
- *Thực thi* test vẫn dồn về nửa sau dự án



# Mô hình Iterative / Incremental

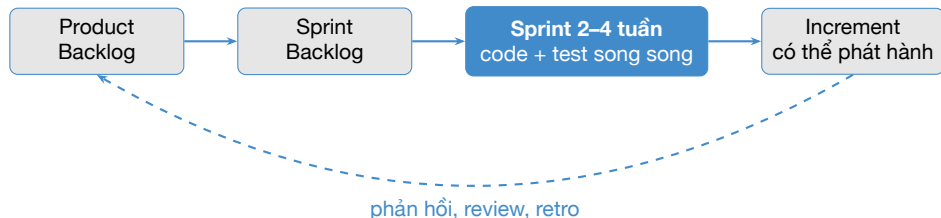


- Hệ thống được xây theo **từng phần tăng trưởng** (increment); mỗi vòng lặp là một “Waterfall thu nhỏ” **có kiểm thử riêng**.
- Kiểm thử diễn ra **nhiều lần, sớm hơn** so với Waterfall → phản hồi nhanh về chất lượng.

## Hệ quả quan trọng

Chức năng của vòng lặp trước phải được **kiểm thử lại** ở vòng lặp sau (regression) — vì code mới có thể phá vỡ code cũ. Regression tăng dần ⇒ bắt buộc **tự động hóa**.

# Agile / Scrum



## Kiểm thử trong Scrum

- Không có “pha test” riêng — test là một phần của **Definition of Done** mỗi user story
- Tester ngồi **trong** team, tham gia từ lúc viết acceptance criteria
- Tự động hóa là nền tảng (unit, API, E2E chạy mỗi lần push)

### Whole-team quality

Developer viết unit test, tester thiết kế test hộp đen và tự động hóa E2E, cả team chịu trách nhiệm chất lượng — không “ném qua tường”.

# Vị trí hoạt động kiểm thử trong từng mô hình

| Mô hình     | Kiểm thử diễn ra khi nào?                                         | Rủi ro / lưu ý                                     |
|-------------|-------------------------------------------------------------------|----------------------------------------------------|
| Waterfall   | Một pha riêng, sau lập trình                                      | Lỗi phát hiện muộn, test bị cắt xén khi trễ hạn    |
| V-model     | <i>Thiết kế</i> test từ đầu mỗi pha; <i>thực thi</i> ở nhánh phải | Vẫn cứng nhắc trước thay đổi yêu cầu               |
| Iterative   | Cuối mỗi vòng lặp                                                 | Khối lượng regression tăng dần theo vòng lặp       |
| Agile/Scrum | Liên tục trong Sprint, mọi story                                  | Đòi hỏi tự động hóa và kỹ năng kỹ thuật của tester |

## Nhận xét

Xu hướng chung của lịch sử các mô hình: kiểm thử dịch chuyển **từ cuối dự án về đầu dự án** — đó chính là *shift-left*.

# Shift-Left Testing

Yêu cầu

Thiết kế

Lập trình

Kiểm thử

Vận hành

←  
đẩy hoạt động kiểm thử sang TRÁI

## Shift-left nghĩa là

- Review đặc tả, review thiết kế (kiểm thử tĩnh)
- Viết acceptance criteria *trước khi* code
- Unit test viết cùng lúc với code (TDD)
- Test tự động chạy ngay mỗi lần commit (CI)

### Trên DigiShield

Mỗi lần build, `./gradlew check` chạy toàn bộ unit test **và** chặn merge nếu độ phủ dòng dưới ngưỡng JaCoCo — lỗi bị chặn ngay tại bàn dev, không đợi đến pha test.

# Quy trình phát triển của DigiShield — hệ thống thực hành

## Bối cảnh

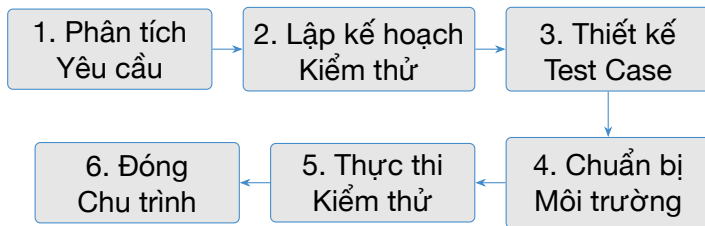
DigiShield (“Lá Chắn Số”) — nền tảng chống lừa đảo/phishing: đào tạo nhận thức, mô phỏng tấn công, tiếp nhận báo cáo, chấm điểm rủi ro, can thiệp giao dịch đáng ngờ. Java 25 + Spring Boot, kiến trúc modular monolith (Spring Modulith, 9 module) tại `modules/`.

Hai tầng kiểm thử được cấu hình sẵn bằng JVM Test Suites:

```
./gradlew test           # unit test (*Test.java),  
                        # không cần Docker  
./gradlew integrationTest # integration (*IT.java),  
                        # cần Docker (Testcontainers)
```

Hiện có **398 test / 50 class** (JUnit 5, Mockito, AssertJ, Testcontainers). Đây là “hiện trường” để ta quan sát mọi khái niệm của buổi học hôm nay.

# STLC — Software Testing Life Cycle



## Entry / Exit criteria [4]

Mỗi giai đoạn có **Entry criteria** (điều kiện để bắt đầu) và **Exit criteria** (điều kiện để kết thúc). Chúng là “cổng chất lượng” ngăn việc nhảy cóc — ví dụ: không thực thi test khi môi trường chưa sẵn sàng.

Giai đoạn 3 và 4 có thể tiến hành *song song*.

# Giai đoạn 1: Phân tích yêu cầu kiểm thử

**Hoạt động:** đọc đặc tả/user story, xác định *cái gì* cần kiểm thử (testable requirements), đặt câu hỏi làm rõ, nhận diện yêu cầu phi chức năng.

## Entry criteria

- Có tài liệu yêu cầu / user story
- Tiếp cận được BA/PO để hỏi đáp
- Có tài liệu kiến trúc (nếu đã có)

## Exit criteria

- Danh sách **testable requirements**; RTM khởi tạo
- Đánh giá khả năng tự động hóa
- Các câu hỏi mở đã được giải đáp

## DigiShield

Đọc yêu cầu “can thiệp giao dịch đáng ngờ”: khi nào PAUSE, WARN, ALLOW?  
Nguồn watchlist là gì? → danh sách hạng mục cần kiểm thử cho module interception.

## Giai đoạn 2: Lập kế hoạch kiểm thử

**Hoạt động:** viết Test Plan — xác định phạm vi, cách tiếp cận, ước lượng công sức, phân công vai trò, chọn công cụ, phân tích rủi ro, lập lịch.

### Entry criteria

- Tài liệu phân tích yêu cầu kiểm thử (GĐ 1)
- RTM khởi tạo
- Thông tin nguồn lực, ngân sách

### Exit criteria

- Test Plan được duyệt/ký
- Ước lượng công sức được chốt
- Vai trò, trách nhiệm đã phân công

### DigiShield

Test Plan cho module interception (mẫu rút gọn ở phần sau): phạm vi = evaluate + watchlist; công cụ = JUnit/Mockito, Postman; tiêu chí pass/fail; lịch theo tuần.



# Giai đoạn 3: Thiết kế test case

**Hoạt động:** viết test case chi tiết (ID, bước, dữ liệu, kết quả mong đợi), chuẩn bị test data, cập nhật RTM (yêu cầu ↔ test case), review chéo test case.

## Entry criteria

- Test Plan đã duyệt
- Yêu cầu đủ chi tiết để thiết kế

## Exit criteria

- Test case + test data được review, ký duyệt
- RTM phủ 100% yêu cầu trong phạm vi

## DigiShield

Từ quy tắc `onCall && newPayee && watchlistHit` → PAUSE thiết kế đủ test case phủ các nhánh PAUSE/WARN/ALLOW — kỹ thuật cụ thể (bảng quyết định, độ phủ nhánh) học ở Buổi 3–4 và 8.

# Giai đoạn 4: Chuẩn bị môi trường kiểm thử

**Hoạt động:** dựng môi trường (server, DB, dữ liệu demo), cài công cụ, smoke test xác nhận môi trường dùng được.

## Entry criteria

Có thiết kế môi trường; có bản build cài đặt được; test data sẵn sàng.

## Exit criteria

Môi trường hoạt động, smoke test pass; test data đã nạp.

Smoke test: mở `http://localhost:8080/api/v1` — dữ liệu demo (6 user seed, vd. `admin@coquan.gov.vn`) tự nạp bởi `DevDataSeeder`.

## DigiShield: 2 môi trường

```
# Moi truong dev (H2, khong can Docker)
./gradlew :boot:app:bootRun \
  --args='--spring.profiles.active=dev'
```

```
# Moi truong prod-like (Docker)
docker compose -f deploy/compose/\
docker-compose.prodlike.yml up --build
```

# Giai đoạn 5: Thực thi kiểm thử

**Hoạt động:** chạy test case theo kế hoạch, ghi kết quả (pass/fail/blocked), báo defect kèm bằng chứng, re-test sau khi sửa, chạy regression.

## Entry criteria

- Test case + test data sẵn sàng
- Môi trường đã smoke test pass
- Bản build cần test đã triển khai

## Exit criteria

- Toàn bộ test case theo kế hoạch đã thực thi
- Defect đã được log và theo dõi
- Kết quả được ghi vào RTM

## DigiShield

Tự động: `./gradlew test` rồi `./gradlew integrationTest`; thủ công/API: chạy collection Postman trên `http://localhost:8080/api/v1`; lỗi tìm được ghi thành bug report (mẫu ở phần sau).

# Giai đoạn 6: Đóng chu trình kiểm thử

**Hoạt động:** viết Test Summary Report, đánh giá theo exit criteria của cả chu trình, họp rút kinh nghiệm (lessons learned), lưu trữ testware để tái sử dụng.

## Entry criteria

- Thực thi đã hoàn tất
- Kết quả test và danh sách defect đầy đủ

## Exit criteria

- Test Summary Report được duyệt
- Defect còn mở có phương án xử lý
- Testware đã bàn giao/lưu trữ

## Nội dung Test Summary Report

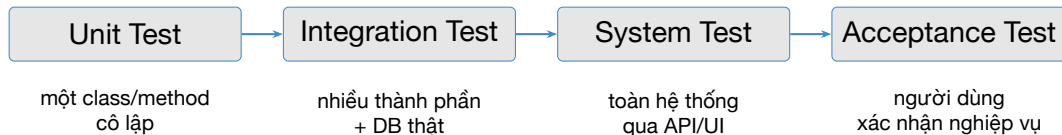
Số test case thực thi, tỷ lệ pass/fail, độ phủ đạt được (JaCoCo), defect theo mức nghiêm trọng, rủi ro còn lại, **khuyến nghị release hay không**.

# Tổng hợp STLC: đầu vào — đầu ra

| Giai đoạn              | Đầu vào chính              | Đầu ra chính                             |
|------------------------|----------------------------|------------------------------------------|
| 1. Phân tích yêu cầu   | Đặc tả, user story         | Danh sách yêu cầu kiểm thử, RTM khởi tạo |
| 2. Lập kế hoạch        | Kết quả GĐ 1, nguồn lực    | Test Plan, ước lượng, phân công          |
| 3. Thiết kế test case  | Test Plan                  | Test case, test data, RTM đầy đủ         |
| 4. Chuẩn bị môi trường | Thiết kế môi trường, build | Môi trường sẵn sàng (smoke pass)         |
| 5. Thực thi            | Test case, môi trường      | Kết quả test, bug report                 |
| 6. Đóng chu trình      | Kết quả thực thi           | Test Summary Report, lessons learned     |

**Ghi nhớ:** STLC lồng **bên trong** SDLC — trong Agile, cả 6 giai đoạn diễn ra *thu nhỏ* trong từng Sprint. BTL cũng đi đúng 6 bước này.

# Bốn mức kiểm thử



## Nguyên tắc phân mức [4, 2]

Mỗi mức có **đối tượng**, **test basis** và **người chịu trách nhiệm** riêng (đúng tinh thần V-model). Lỗi nên được bắt ở mức **thấp nhất có thể** — rẻ hơn và dễ chẩn đoán hơn.

Bốn slide tiếp theo: ánh xạ từng mức vào DigiShield.

# Mức 1: Unit Test trên DigiShield

## Đặc điểm

- Test **một class/method**, mock mọi dependency
- Nằm trong `modules/*/src/test`, tên `*Test.java`
- JUnit 5 + Mockito + AssertJ; hiện có **398 test**
- Không cần Docker, chạy trong mili-giây

```
./gradlew test  
./gradlew jacocoTestReport
```

## InterceptionServiceImplTest (trích)

```
@ExtendWith(MockitoExtension.class)  
class InterceptionServiceImplTest {  
    @Mock AccountWatchEntryRepository watchRepo;  
    @Mock InterventionEventRepository eventRepo;  
    @InjectMocks InterceptionServiceImpl svc;  
  
    @Test  
    void evaluate_allSignals_pauses() {  
        // Arrange: dich den nam trong watchlist  
        when(watchRepo  
            .findFirstByTenantIdAndValueOrderByAddedAtDesc(  
                TENANT_ID, DEST))  
            .thenReturn(Optional.of(watchHit));  
        // Act  
        var d = svc.evaluate(  
            new EvaluateRequest(userId, amount,  
                DEST, true, true));  
        // Assert: nhanh PAUSE duoc chon  
        assertThat(d.decision())  
            .isEqualTo("pause");  
    }  
}
```

# Mức 2: Integration Test trên DigiShield

## Đặc điểm

- Nhiều thành phần làm việc với nhau: controller + service + **PostgreSQL thật** (Testcontainers)
- Nằm ở boot/app/src/integrationTest, tên \*IT.java
- Cần Docker; chạy trong vài giây tới vài chục giây

```
./gradlew integrationTest
```

## Ví dụ thật trong repo

- TenantIsolationIT: chứng minh Row-Level Security cách ly dữ liệu giữa 2 tenant (fail-closed)
- InterceptionControllerIT: gọi HTTP thật vào endpoint evaluate
- SimulationModuleIT, RlsCoverageIT

Câu hỏi mức này trả lời: “các mảnh ghép có khớp nhau không?” — điều unit test với mock **không thể** phát hiện.



# Mức 3: System Test trên DigiShield

Kiểm thử **toàn hệ thống đã triển khai**, từ bên ngoài, dựa trên đặc tả — gồm cả **phi chức năng** (hiệu năng, bảo mật, dễ dùng).

## Qua API — Postman (Buổi 9)

Base URL: `localhost:8080/api/v1`

- Import spec  
`DigiShield_openapi.yaml`
- POST  
`/interventions/evaluate`
- GET `/account-watchlist/check?value=...`
- Test tiêu cực: **401** chưa đăng nhập, **403** sai role

## Qua UI — Selenium (Buổi 10–11)

Frontend React chạy ở  
`localhost:5173`

- Đăng nhập tại `/login` (select `#login-role`, input `#login-email`, nút `#login-submit`)
- Luồng SOC Inbox, Watchlist

# Mức 4: Acceptance Test (UAT) trên DigiShield

Người dùng cuối / khách hàng thực thi các **kịch bản nghiệp vụ** để quyết định *chấp nhận* sản phẩm (test basis: yêu cầu người dùng).

## Kịch bản UAT: chuyên viên phân tích (analyst) triage báo cáo phishing

- 1 Người dùng cuối gửi báo cáo phishing (POST / reports/phishing hoặc UI)
- 2 Đăng nhập bằng `analyst@coquan.gov.vn` (role ANALYST)
- 3 Analyst mở SOC Inbox, xem chi tiết báo cáo và nhấn AI gợi ý
- 4 **Xác nhận** → CONFIRMED, phát cảnh báo; **bác bỏ** → DISMISSED
- 5 Chuyển báo cáo đã xác nhận thành tình huống đào tạo (convert-to-training)

**Tiêu chí chấp nhận:** analyst tự hoàn thành luồng, dữ liệu nhất quán,  $\leq 2$  phút/báo cáo.

# Bảng ánh xạ: mức kiểm thử ↔ DigiShield

| Mức         | Trên DigiShield                                                                                | Công cụ / vị trí                                       | Buổi học      |
|-------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------|
| Unit        | <code>./gradlew test</code> — 398 test trong modules/*                                         | JUnit 5 + Mockito, *Test.java                          | 3–6           |
| Integration | <code>./gradlew integrationTest</code>                                                         | Testcontainers, boot/app/src/integrationTest, *IT.java | 7             |
| System      | API test trên <code>localhost:8080/api/v1</code> ;<br>UI test trên <code>localhost:5173</code> | Postman/Newman; Selenium                               | 9–11          |
| Acceptance  | UAT theo kịch bản nghiệp vụ (analyst triage báo cáo phishing)                                  | Kịch bản UAT, checklist                                | 12 (demo BTL) |

## Ghi nhớ cho BTL

Bài tập lớn yêu cầu nhóm chạy đủ **cả 4 mức** này trên module mình chọn.

# Kiểm thử chức năng và phi chức năng

## Chức năng (functional) — “làm gì”

Kiểm tra hệ thống làm **đúng việc** được đặc tả.

- Evaluate trả về PAUSE khi đủ 3 tín hiệu rủi ro
- Triage xác nhận → trạng thái CONFIRMED
- Login sai mật khẩu → từ chối

## Phi chức năng (non-functional) — “tốt thế nào”

Kiểm tra **chất lượng** khi thực hiện việc đó.

- Hiệu năng: evaluate phản hồi < 200ms
- Bảo mật: LEARNER gọi API của ANALYST bị 403; cách ly dữ liệu giữa các tenant
- Usability: form báo cáo dễ dùng trên di động

**Ngoài ra:** kiểm thử **cấu trúc** (white-box — độ phủ lệnh/nhánh, Buổi 3–4) và kiểm thử **liên quan thay đổi** (re-test, regression — slide sau).

# Re-test (kiểm thử lại) và Regression (hồi quy)

## Re-test (confirmation testing) [4]

Chạy lại **đúng test case đã fail** sau khi lỗi được sửa, để xác nhận lỗi **đã hết**.

- Cùng môi trường, cùng dữ liệu, cùng bước
- Bug chuyển từ Fixed → Verified

## Regression testing

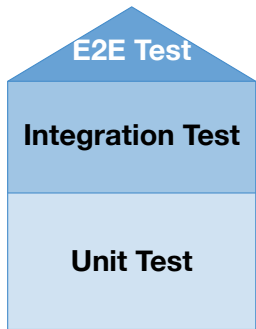
Chạy lại **các test case đã pass** quanh vùng thay đổi, để xác nhận thay đổi **không phá** chức năng cũ.

- Tập test tích lũy theo thời gian
- Bắt buộc tự động hóa, gắn vào CI

## Ví dụ DigiShield

Dev sửa lỗi trong `evaluate()` khiến WARN trả về sai. **Re-test**: chạy lại test WARN đã fail. **Regression**: chạy lại **toàn bộ** `./gradlew test` (398 test) — đảm bảo các nhánh PAUSE/ALLOW và 8 module còn lại không bị ảnh hưởng.

# Testing Pyramid



| Tầng  | Số lượng   | Tốc độ | Chi phí |
|-------|------------|--------|---------|
| Unit  | Nhiều nhất | ms     | Thấp    |
| Integ | Vừa phải   | giây   | Vừa     |
| E2E   | Ít nhất    | phút   | Cao     |

## Tỉ lệ khuyến nghị [7]:

70% Unit : 20% Integration : 10%  
E2E

Càng lên cao: càng **giống người dùng thật** nhưng càng **chậm, đắt, mong manh** (flaky) và khó chẩn đoán khi fail.

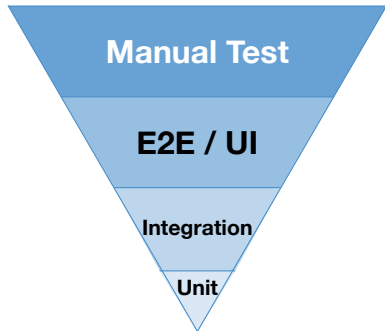
# Pyramid soi vào DigiShield hiện tại

| Tầng        | Hiện trạng DigiShield                                                     | Nhận xét                                         |
|-------------|---------------------------------------------------------------------------|--------------------------------------------------|
| Unit        | 398 test, 50 class, chạy bằng<br>./gradlew test                           | Đáy tháp đã có nền                               |
| Integration | 10 file *IT.java (38<br>test, Testcontainers): RLS,<br>controller, module | Mỏng hơn — đúng hình<br>tháp                     |
| E2E / UI    | Selenium :e2e mới có <b>1 kịch<br/>bản</b>                                | <b>Sinh viên mở rộng<br/>bằng Selenium (BTL)</b> |

## Bài học

Một dự án thật hiếm khi có tháp “đẹp” ngay từ đầu. Việc của tester: **nhận diện lỗ hổng theo tầng** và bổ sung có chiến lược — không phải viết thật nhiều E2E cho mọi thứ.

# Anti-pattern: Ice-Cream Cone



**Tháp lộn ngược [7]:** nhiều test thủ công/E2E, rất ít unit test.

## Hậu quả

- Bộ test chạy **hàng giờ**, phản hồi chậm
- E2E flaky → mất niềm tin vào kết quả test
- Lỗi phát hiện muộn, khó khoanh vùng nguyên nhân
- Chi phí bảo trì test tăng vọt theo UI

## Cảnh báo cho BTL

Nhóm chỉ nộp Selenium E2E, thiếu unit test  
→ **mất điểm** theo rubric.



# Test Plan theo IEEE 829

## Định nghĩa [6]

Test Plan là tài liệu mô tả **phạm vi, cách tiếp cận, tài nguyên và lịch trình** của hoạt động kiểm thử — sản phẩm của Giai đoạn 2 STLC.

### Các mục chính (IEEE 829):

- Test plan identifier; Introduction
- Test items — phần mềm/module nào
- Features to be tested / **not** to be tested
- Approach — phương pháp, kỹ thuật, công cụ

### (tiếp)

- Item pass/fail criteria
- Suspension / resumption criteria
- Test deliverables; Test environment
- Responsibilities; Schedule; **Risks** and contingencies

Hai slide sau: Test Plan **rút gọn** cho module interception của DigiShield — dùng làm mẫu cho BTL.

# Ví dụ: Test Plan module interception (1/2)

## TP-INT-01 — Phạm vi (Test items & Features)

**Trong phạm vi:** `InterceptionServiceImpl.evaluate()` (quyết định PAUSE/WARN/ALLOW); phân trang `listInterventions(page, size)`; API `/interventions/evaluate`, `/account-watchlist` (GET/POST), `/account-watchlist/check`; phân quyền ANALYST.

**Ngoài phạm vi:** hiệu năng dưới tải lớn; tích hợp SMS/AWS SNS thật.

## Rủi ro sản phẩm (chọn lọc)

- **R1 — Cao:** ALLOW sai một giao dịch lừa đảo → người dùng mất tiền
- **R2 — Cao:** PAUSE sai giao dịch hợp lệ → mất niềm tin, khiếu nại
- **R3 — Cao:** rò rỉ watchlist giữa các tenant (multi-tenant)
- **R4 — Trung bình:** tham số phân trang biên (0, 100, 101) gây lỗi/tràn
- **R5 — Trung bình:** LEARNER truy cập được API dành cho ANALYST

⇒ Ưu tiên test theo rủi ro: R1–R3 phủ dày nhất.

# Ví dụ: Test Plan module interception (2/2)

## Approach & tiêu chí Pass/Fail

**Approach:** unit (JUnit+Mockito, bảng quyết định cho evaluate); integration (InterceptionControllerIT, Testcontainers); API (Postman, gồm test tiêu cực 401/403); E2E (Selenium: luồng Watchlist).

**Pass:** 100% test case đã thiết kế được thực thi; tỷ lệ pass  $\geq 95\%$ ; độ phủ nhánh của evaluate() = 100%; **không còn defect Critical/High mở**. **Suspension:** dừng khi môi trường sập hoặc  $>30\%$  test blocked.

## Môi trường

- Unit: JDK, không cần Docker
- IT: Docker + Testcontainers
- API/E2E: backend dev (H2, cổng 8080) + frontend (cổng 5173)

## Lịch (gắn timeline BTL)

- Tuần 1: test plan + danh sách hạng mục
- Tuần 2–4: thiết kế + unit test
- Tuần 5–6: API test Postman
- Tuần 7–8: Selenium + báo cáo

# Test Strategy so với Test Plan

| Thuộc tính   | Test Strategy                                                                   | Test Plan                                                     |
|--------------|---------------------------------------------------------------------------------|---------------------------------------------------------------|
| Phạm vi      | Toàn tổ chức / dự án                                                            | Một vòng kiểm thử / module cụ thể                             |
| Người viết   | Test Manager / QA Lead                                                          | Test Lead / Engineer                                          |
| Nội dung     | Nguyên tắc, công cụ, quy trình chung                                            | Chi tiết: scope, resource, schedule                           |
| Tính ổn định | Ít thay đổi                                                                     | Cập nhật theo từng release                                    |
| Ví dụ        | “Mọi endpoint DigiShield phải có Postman test tiêu cực 401/403 trước khi merge” | “Vòng này: test evaluate + watchlist của module interception” |

**Quan hệ:** Strategy là “hiến pháp” chất lượng; mỗi Test Plan phải **tuân theo** strategy và cụ thể hóa nó cho một phạm vi, một giai đoạn.

# Traceability Matrix (RTM): yêu cầu ↔ test case

**Mục đích:** chứng minh **mọi yêu cầu đều có test** và mọi test đều truy về một yêu cầu; đánh giá tác động khi yêu cầu thay đổi.

## Ví dụ RTM — chức năng triage báo cáo phishing (DigiShield)

| Req ID   | TC ID    | Yêu cầu                                                  | Mức         | Trạng thái |
|----------|----------|----------------------------------------------------------|-------------|------------|
| RQ-TR-01 | TC-TR-01 | Xác nhận threat → trạng thái CONFIRMED, nhãn AI = THREAT | Unit        | Pass       |
| RQ-TR-02 | TC-TR-02 | Xác nhận threat → phát PhishingReportConfirmedEvent      | Unit        | Pass       |
| RQ-TR-03 | TC-TR-03 | Bác bỏ → trạng thái DISMISSED, nhãn CLEAN                | Unit        | Pass       |
| RQ-TR-04 | TC-TR-04 | Không triage được báo cáo của tenant khác                | Integration | Pass       |
| RQ-TR-05 | TC-TR-05 | LEARNER gọi POST /reports/phishing/{id}/triage → 403     | API         | Chưa chạy  |

RTM là sản phẩm bắt buộc trong báo cáo BTL (khởi tạo GĐ 1, hoàn thiện GĐ 3, cập nhật GĐ 5).

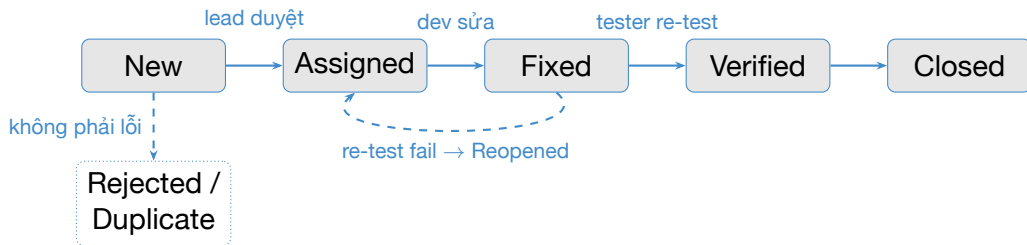
# Vai trò và trách nhiệm trong nhóm kiểm thử

| Vai trò                  | Trách nhiệm chính                                            |
|--------------------------|--------------------------------------------------------------|
| Test Manager / QA Lead   | Test strategy, ngân sách, nhân sự, báo cáo lên quản lý dự án |
| Test Lead                | Test plan, phân công, theo dõi tiến độ, tổng hợp báo cáo     |
| Test Analyst / Designer  | Phân tích yêu cầu, thiết kế test case, test data, RTM        |
| Tester (manual)          | Thực thi test case, log defect, re-test                      |
| Automation Tester / SDET | Framework test tự động, JUnit/Postman/Selenium, tích hợp CI  |
| Developer                | Unit test cho code của mình, sửa defect                      |
| Product Owner / User     | Acceptance criteria, thực hiện UAT                           |

## Áp vào nhóm BTL (3–5 SV)

Nhóm tự phân vai: 1 bạn kiêm Test Lead, còn lại chia nhau Designer / Automation / API / E2E. **Mọi thành viên đều phải viết test.**

# Defect Lifecycle cơ bản



- **Severity** (mức nghiêm trọng với hệ thống)  $\neq$  **Priority** (mức khẩn với kế hoạch) — lỗi chính tả trên trang chủ: severity thấp, priority cao.
- Re-test chuyển Fixed  $\rightarrow$  Verified; regression đảm bảo bản sửa không phá chỗ khác.

# Bug report mẫu (lỗi giả định trên DigiShield)

|                            |                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <b>ID</b>                  | BUG-DS-042                                                                                         |
| <b>Tiêu đề</b>             | Check watchlist với value rỗng trả 200 thay vì 400                                                 |
| <b>Module</b>              | interception — GET /account-watchlist/check?value=                                                 |
| <b>Môi trường</b>          | Backend dev (H2), commit 7ab8bc6, đăng nhập role ANALYST                                           |
| <b>Các bước</b>            | 1) Đăng nhập analyst@coquan.gov.vn; 2) Gửi GET /api/v1/account-watchlist/check?value= (chuỗi rỗng) |
| <b>Kết quả thực tế</b>     | HTTP 200, body {"inWatchlist": false}                                                              |
| <b>Kết quả mong đợi</b>    | HTTP 400 — tham số value bắt buộc, không được rỗng                                                 |
| <b>Severity / Priority</b> | Medium / Medium                                                                                    |
| <b>Đính kèm</b>            | Ảnh chụp Postman, log request                                                                      |
| <b>Trạng thái</b>          | New                                                                                                |

Gốc rễ: DigiShield validate **thủ công** trong service → dễ bỏ sót ca biên — “vùng đất màu mỡ” để các nhóm săn lỗi thật.



# Bài tập lớn — A1.2, chiếm 30% điểm học phần

## Tổ chức

Nhóm **3–5 sinh viên**. Đăng ký nhóm + module **ngay hôm nay** (phần thực hành). Mỗi nhóm một module, không trùng nhau nếu lớp đủ module.

## Đề bài

Thực hiện **trọn vẹn quy trình kiểm thử (STLC)** cho:

- **Phương án A (khuyến nghị)**: một module của DigiShield — gợi ý: interception, reporting, ai, learning, simulation; hoặc
- **Phương án B**: ứng dụng do nhóm tự chọn, **phải được GV duyệt** trước Buổi 3 (đủ độ phức tạp: có API, có UI, có logic phân nhánh).

Điểm nhóm  $\times$  hệ số đóng góp cá nhân (peer review + vấn đáp khi demo).

# Bài tập lớn — Sản phẩm phải nộp

- 1 **Test Plan** theo khung IEEE 829 rút gọn (như mẫu interception hôm nay)
- 2 **Thiết kế test case**: hộp đen (EP, BVA, bảng quyết định — Buổi 8–9) và hộp trắng (độ phủ nhánh — Buổi 3–4), kèm RTM
- 3 **Unit test tự động**: JUnit 5 + Mockito chạy được bằng `./gradlew test` (Buổi 5–7), kèm báo cáo độ phủ JaCoCo
- 4 **API test**: Postman collection + chạy Newman, có test tiêu cực 401/403 (Buổi 9)
- 5 **E2E test**: kịch bản Selenium trên frontend `localhost:5173` (Buổi 10–11)
- 6 **Báo cáo tổng kết** (Test Summary Report) + **demo bảo vệ tại Buổi 12**

## Lưu ý

Sản phẩm 1–2 làm được ngay sau hôm nay; sản phẩm 3–5 tích lũy dần theo nội dung từng buổi — **đừng dồn về cuối kỳ.**

# Gợi ý module DigiShield cho các nhóm

| Module       | Nghịệp vụ                                  | Điểm hay để kiểm thử                                                   |
|--------------|--------------------------------------------|------------------------------------------------------------------------|
| interception | Can thiệp giao dịch đáng ngờ, watchlist    | Logic PAUSE/WARN/ALLOW nhiều nhánh; BVA phân trang; phân quyền ANALYST |
| reporting    | Tiếp nhận + triage báo cáo phishing        | State machine của triage; ageLabel hợp cho EP/BVA; cách ly tenant      |
| ai           | Phân loại/moderate nội dung, sinh template | StubAiClient dễ test; validate thủ công; fallback Claude → Stub        |
| learning     | Khóa học, quiz nhận thức                   | Luồng học + quiz trên UI (Selenium); auto-enroll qua sự kiện           |
| simulation   | Chiến dịch phishing mô phỏng               | Wizard tạo campaign (UI); sự kiện click/report; thống kê               |

Mỗi module đều có: service có logic phân nhánh (unit), REST API (Postman), trang UI liên quan (Selenium) — đủ nguyên liệu cho cả 6 sản phẩm BTL.

# Timeline Bài tập lớn theo 12 buổi

| Buổi | Mốc              | Nội gì                                                    |
|------|------------------|-----------------------------------------------------------|
| 2    | Lập nhóm         | Danh sách nhóm + module đăng ký (cuối buổi)               |
| 3    | Phạm vi          | Danh sách 10 hạng mục cần kiểm thử (đã sửa sau thực hành) |
| 4    | <b>Test Plan</b> | Test Plan v1 theo IEEE 829 rút gọn                        |
| 6    | Hộp trắng        | Test case hộp trắng + unit test JUnit đầu tiên            |
| 7    | <b>Unit/IT</b>   | Bộ JUnit + Mockito hoàn chỉnh, báo cáo JaCoCo             |
| 9    | <b>API</b>       | Test case hộp đen + Postman/Newman collection             |
| 11   | <b>E2E</b>       | Kịch bản Selenium chạy được + báo cáo nháp                |
| 12   | <b>Bảo vệ</b>    | Báo cáo cuối (PDF) + demo + vấn đáp                       |

## Nguyên tắc chấm mốc

Mỗi mốc nộp trễ: -10% điểm thành phần tương ứng. Mốc Buổi 4, 7, 9, 11, 12 là **bắt buộc** — thiếu 2 mốc bắt buộc  $\Rightarrow$  không được bảo vệ.

# Rubric chấm Bài tập lớn (thang 10)

| Tiêu chí                       | Điểm | Yêu cầu đạt mức tối đa                                                                |
|--------------------------------|------|---------------------------------------------------------------------------------------|
| Test Plan + RTM                | 1.5  | Đủ mục IEEE 829, rủi ro có ưu tiên, RTM phủ 100% phạm vi                              |
| Thiết kế test case đen + trắng | 2.5  | Áp dụng đúng kỹ thuật (EP/BVA/bảng quyết định; độ phủ nhánh), có ca tiêu cực          |
| JUnit + Mockito (+ JaCoCo)     | 2.0  | Chạy pass bằng <code>./gradlew test</code> ; độ phủ nhánh phần logic chọn $\geq 80\%$ |
| Postman / Newman               | 1.5  | Collection chạy bằng Newman; có test 401/403 và dữ liệu biên                          |
| Selenium E2E                   | 1.5  | $\geq 2$ kịch bản chạy ổn định, có wait hợp lý                                        |
| Báo cáo + demo + vấn đáp       | 1.0  | Test Summary Report; trả lời được về mọi phần của nhóm                                |

Tìm được **lỗi thật** trong DigiShield kèm bug report chuẩn: **+0.5 điểm thưởng**/lỗi (tối đa +1.0).

# Thông báo: Kiểm tra viết (10%) — đầu Buổi 3

## Bài kiểm tra viết giữa kỳ — A1.1 (10% điểm học phần)

**Thời điểm:** 30 phút đầu Buổi 3. **Hình thức:** tự luận ngắn, không sử dụng tài liệu.

**Phạm vi: Chương 1 + Chương 2**, trọng tâm:

- Định nghĩa kiểm thử; Error — Defect — Failure; chi phí lỗi
- 7 nguyên tắc kiểm thử (phát biểu + ví dụ)
- Vẽ và giải thích mô hình V; so sánh vị trí kiểm thử trong Waterfall/Agile
- 6 giai đoạn STLC + entry/exit criteria
- 4 mức kiểm thử; re-test vs regression; testing pyramid
- Các mục chính của Test Plan theo IEEE 829

Dạng câu hỏi tham khảo: “Vẽ mô hình V và chỉ ra mức kiểm thử tương ứng với Thiết kế Module”; “Phân biệt re-test và regression, cho ví dụ trên DigiShield”.

# Thực hành trên lớp (theo nhóm BTL vừa lập)

## Phần 1 — Danh sách 10 hạng mục cần kiểm thử (25 phút)

Với module nhóm đã chọn, liệt kê **10 hạng mục** (test items/features):

- Mỗi hạng mục: tên chức năng, endpoint/class liên quan, mức kiểm thử dự kiến (unit/integration/system), rủi ro nếu hỏng (Cao/TB/Thấp)
- Tham khảo mã nguồn trong `modules/<tên-module>` và spec `../docs/DigiShield_openapi.yaml` (thư mục cha của repo)

## Phần 2 — Phác thảo Test Plan 1 trang (25 phút)

Theo khung: Phạm vi (in/out) — Approach — Rủi ro (3–5 dòng có ưu tiên) — Tiêu chí pass/fail — Môi trường — Lịch theo timeline BTL.

10 phút cuối: 2–3 nhóm trình bày nhanh, cả lớp phản biện (“hạng mục nào thiếu? rủi ro nào xếp sai?”).

# Bài nộp — Deadline: trước Buổi 3 (1/2)

## Nộp theo nhóm (1 file PDF)

- 1 Danh sách nhóm: họ tên, MSSV, vai trò dự kiến (Test Lead / Designer / Automation / API / E2E)
- 2 Module đăng ký (hoặc đề xuất ứng dụng tự chọn kèm mô tả để GV duyệt)
- 3 Danh sách 10 hạng mục cần kiểm thử (hoàn thiện từ phần thực hành)
- 4 Test Plan 1 trang (hoàn thiện từ phần thực hành)



# Bài nộp — Deadline: trước Buổi 3 (2/2)

## Nộp cá nhân

Vẽ lại mô hình V (tay hoặc công cụ) và ghi chú: với module nhóm em, *mỗi mức kiểm thử sẽ dùng công cụ gì trên DigiShield?*

## Định dạng

Nhom<X>\_Buoi02.pdf và HoTen\_MaSV\_Buoi02.pdf. Đồng thời ôn Chương 1+2 cho bài kiểm tra viết đầu Buổi 3.

# Tóm tắt buổi 2

## Lý thuyết đã học:

- Vị trí kiểm thử trong Waterfall, V-model, Iterative, Agile; shift-left
- STLC: 6 giai đoạn + entry/exit criteria
- 4 mức kiểm thử ánh xạ vào DigiShield
- Chức năng/phi chức năng; re-test/regression
- Testing pyramid và ice-cream cone
- Test Plan IEEE 829, Test Strategy, RTM
- Vai trò nhóm kiểm thử; defect lifecycle

## Việc phải làm:

- Nộp danh sách nhóm + module BTL
- Hoàn thiện 10 hạng mục + test plan 1 trang
- Ôn Chương 1+2 → **kiểm tra viết đầu Buổi 3 (10%)**

## Buổi tới (Buổi 3)

Kiểm thử hộp trắng — luồng điều khiển: CFG, độ phủ câu lệnh / nhánh / đường, thực hành trên evaluate() của DigiShield.

# Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., Wiley, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023.
- [5] ISTQB — International Software Testing Qualifications Board.  
<https://istqb.org/>
- [6] IEEE Std 829-2008. *IEEE Standard for Software and System Test Documentation*. IEEE, 2008.
- [7] Martin Fowler. “The Practical Test Pyramid” và các bài viết về kiểm thử.  
<https://martinfowler.com/>